Predicting Shipment ETA and Delay Propagation

Aiqon take-home, revised after a code/methodology audit. **All quantitative results are on synthetic data** and do not measure real-world accuracy. Every number below is read from `report/results.md`, `chain_fix_effect.md` and `chain_dprev_mismatch.md` (generated by the code).

1. Problem framing and definitions

Delay at a milestone = actual − scheduled timestamp (hours). **Final ETA** = scheduled delivery + predicted final delay; a shipment is **"badly delayed"** if final delay > 48 h. Milestones: departure, port arrival, discharge, customs clearance, delivery. The system re-predicts at five **checkpoints** (booked, departed, at port, discharged, customs cleared) using only information available at that time. Missing intermediate timestamps are expected.

2. Data (synthetic) and ground truth

No free public dataset has scheduled-vs-actual times for every stage (the nearest Kaggle shipping set is itself synthetic; real port-call feeds cover one leg; carrier/AIS data is commercial), so I built a **causal simulator** (2,000 shipments, 10 India→Gulf/Singapore/Europe/US routes, 2023–24). Real inputs: port pairs and liner-like transit times, and **real ERA5 daily wind at every port** (Open-Meteo). Everything else (terminal-efficiency tiers loosely inspired by the World Bank CPPI, congestion dynamics, effect sizes, the 48 h threshold, chosen so ~55% of shipments are badly delayed) is *my assumption*. Propagation: $d_k = m_k \cdot d_{k-1} + own_k$ with $m_k \in [0,1]$ the surviving share of upstream delay (slack; at most 24 h recoverable at sea). Modelled behaviours: congestion × inefficiency interaction on berth wait and dwell; missed-berth-window jumps (+24–48 h); customs office hours (Mon–Fri 09–17) and truck dispatch cut-off; random inspections and documentation errors; real-wind weather effects; transshipment; informative missing timestamps. Each driver's hours are logged, so the final delay decomposes *exactly* into per-stage contributions (tested), the ground truth for scoring explanations.

3. Features, checkpoint availability and model

- **Information availability** (enforced in `features.py`, tested): at booking only static fields (route, efficiency, cargo risk, documentation flag) are known; after departure also origin wind and the destination congestion index *of the departure day*; at port arrival the arrival-day congestion and wind; milestone delays only once reached. Anything unobserved is NaN, never a future value or a forecast stand-in. Feature groups: upstream cumulative delay and increments, slack vs berth window, delay × congestion, congestion × (1−efficiency), expected customs-closed and dispatch-cut-off waits from the *expected* clock time of the next stage, missing-stage counts.
- **Chain model** (explicit propagation): per stage, a GBM predicts the stage increment from the previous stage's delay plus that stage's drivers; rolling forward from the last observed milestone gives the final delay. **Direct model**: one GBM over all checkpoints. Per-checkpoint blend weights.
- **Data roles**: train → fit both models; calibration half A → blend weights; calibration half B → split-conformal 80% intervals and isotonic calibration; the test set is never used in fitting. Hyper-parameters were fixed a priori; the only model-side choice made after the audit (chain loss) was checked on the calibration split alone and left at the default.
- **Explanations**: (a) *where hours were lost*: stage increment (realised, or expected for future stages) minus the typical increment; (b) indicative *what-if*: re-roll the chain with one stage set to normal; (c) TreeSHAP top positive driver per stage; plain-English template (no LLM).

4. Audit: what was found and fixed

- **Confirmed: chain train/inference mismatch.** Stage models were trained with every exogenous input (as if at checkpoint 4) but queried at earlier checkpoints with those inputs missing. LightGBM treats NaN as 0 when training had none, so the old chain saw "zero congestion": measured, it predicted a mean port-stage increment of −0.9 h where the true mean is 13.0 h. **Fix**: one stage model per information regime (booked / departed / at-port-or-later), each trained on exactly the features available then. Trade-off: 3× more small models, less data per regime.
- **Confirmed: "booking" congestion was the departure-day value**, not knowable at booking. Now available from the "departed" checkpoint only (renamed `cong_at_departure`).
- **Confirmed: calibration roles overlapped** (blend weights tuned and conformal residuals computed on the same data). Now split in two halves by shipment.
- **Confirmed: weak validation** (actual timeline never checked; CLI accepted incomplete records), **explanation bug** (driver computed with the wrong previous-stage delay for future stages; could name a negative contributor), unclosed file handles, unpinned dependencies, silent overwrite of results. All fixed; results are now archived under `report/archive/` and a run-metadata file records seeds, parameters and package versions.
- **Investigated, real, fix tested and rejected: previous-stage-delay mismatch.** Stage models are trained with the *true* previous-stage delay (teacher forcing) but, beyond the last observed milestone, receive a *rolled-forward prediction*. Measured on the validation split, that input has RMSE 6 / 11 / 30 / 37 h for stages 1–4 against a true standard deviation of 6 / 13 / 37 / 44 h, so far-ahead inputs are barely informative where training saw exact values. I implemented the principled fix (retrain each stage on out-of-fold rolled-forward delays, plus a flag for "previous delay

observed"); on the validation split it was *not* better (chain-only MAE 19.96 h vs 19.3 h, worse at "discharged": 15.9 vs 13.7 h), so the default stays teacher-forced and the option (`Chain(oof_folds=5)`) is off. The mismatch therefore remains a known limitation of the chain model; the blend with the direct model, which has no such issue, limits its impact. Reproduce: `scripts/chain_dprev_mismatch.py` (validation split only).

- **Checked and found sound**: temporal and route splits are isolated (tested); route hold-out excludes held-out routes from training and calibration; features contain no future timestamps (adversarial tests: altering everything after checkpoint *k*, including later wind/congestion and the target, leaves predictions and attributions at *k* unchanged; the test fails if a leak is re-introduced); bootstrap was already paired by shipment. Ablations alter only the direct model.
- **Disclosed methodological caveat**: I chose the attribution accounting ("where hours were lost" rather than the counterfactual view) after seeing fidelity on the test split; this is mitigated by the replication on two fresh simulated worlds (Section 5D) that played no part in that choice.

5. Results (synthetic data)

A. Temporal split (1,200 train / 300 calibration / 500 test shipments; ETA MAE in hours).

MAE (h)	booked	departed	at port	discharged	cust. cleared	overall
Schedule as ETA	52.0	52.0	52.0	52.0	52.0	52.0
Best baseline (last delay + median remaining)	29.0	29.2	25.8	21.9	15.1	24.2
Direct GBM	22.7	22.8	18.8	13.9	7.0	17.0
Chain (propagation) model	27.1	26.7	17.2	12.1	4.3	17.5
Blend (ours), after fixes	**23.1**	**23.8**	**16.9**	**11.8**	**4.2**	**16.0**
Blend, before fixes (old report)	22.6	22.5	16.9	11.7	4.3	15.6
Direct, no interaction/calendar features	25.8	25.7	22.6	21.1	15.3	22.1

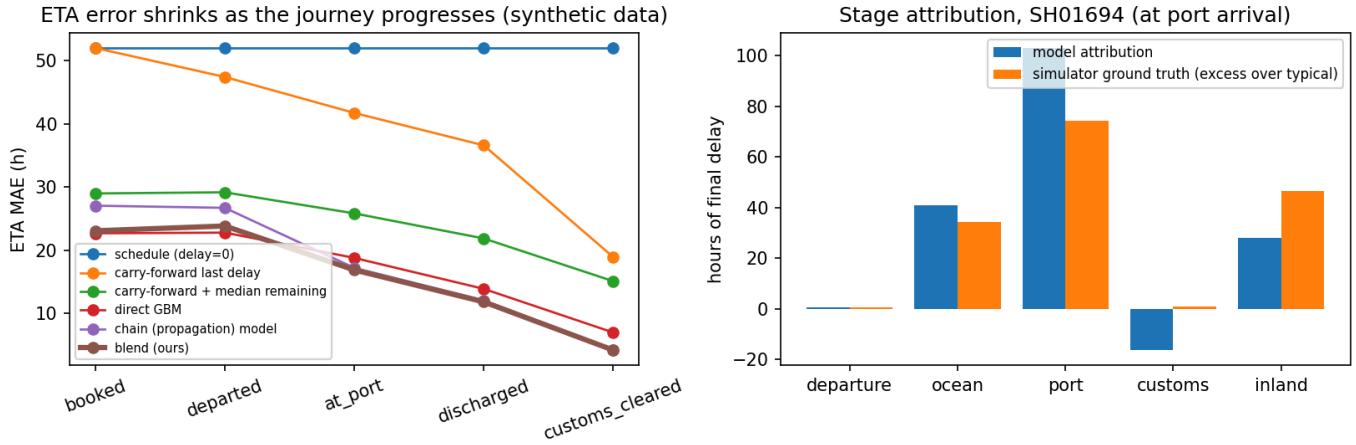


Fig. 1 (synthetic data). Left: ETA MAE by checkpoint. Right: stage attribution for one held-out shipment at port arrival vs simulator ground truth (both as excess over a typical shipment).

- **ETA**: overall blend MAE 16.0 h (95% paired bootstrap CI 14.7–17.3) vs 24.2 h for the best baseline; paired improvement 7.1–9.5 h. **Slight regression vs the old 15.6 h** (old CI 14.5–16.8, overlapping): early checkpoints are 0.5–1.3 h worse because congestion is no longer visible at booking and the chain no longer benefits from the artefact above. The chain-only comparison (`chain_fix_effect.md`) shows the corrected chain is worse at booking (27.1 vs 23.0 h; overall 17.5 vs 16.6 h) and identical from "at port" onward; I kept the corrected version. Removing interaction/calendar features costs the direct model 5.1 h; weather and missingness features each change it by ≤0.1 h here (the simulated weather effect is small).
- **Intervals** (80% nominal): conformal coverage 0.77 / 0.80 / 0.82 / 0.79 / 0.87 by checkpoint (mean 0.81; old 0.82–0.84), width 86 h at booking → 10 h at the end. Un-calibrated quantile-GBM intervals cover 0.75–0.79, so conformalisation helped little on this split.
- **P(delay > 48 h)**: AUC 0.891 (old 0.894), Brier 0.136, ECE 0.035 (old 0.023). The raw GBM classifier is better calibrated (ECE 0.028) than my calibrated blend, so the calibration layer is not shown to help here. AUC by checkpoint 0.82 / 0.81 / 0.86 / 0.91 / 0.99.
- **Explanation fidelity** (stage-level, excess over typical; chance top-1 = 0.20): Spearman 0.29 / 0.31 / 0.50 / 0.64 / 0.83 and top-1 agreement 0.48 / 0.50 / 0.58 / 0.71 / 0.84 from booked to customs-cleared (old: Spearman 0.31–0.83, top-1 0.51–0.84). Explanations are weak before departure, as expected. The what-if view agrees less (Spearman 0.18–0.45) because it credits knock-on effects to their cause, which ground-truth accounting books where they occur; treat its magnitudes as indicative.

B. 300-shipment subset (180/45/75; the assignment size). Blend MAE 16.7 h (CI 13.5–21.1) vs 24.2 h baseline (CI 20.8–28.5), paired difference 5.1–10.0 h (old: 16.9 h). With about 22 shipments per calibration half, coverage swings between 0.69 and 0.92 across checkpoints and the calibrated direct classifier is poor (ECE 0.159): small-sample calibration is unreliable.

C. Route hold-out (Nhava Sheva>Hamburg and Chennai>Los Angeles unseen; 1,289/322/389). Blend MAE 22.3 h (CI 20.4–24.4) vs 33.0 h baseline (paired difference 9.1–12.2 h; old 22.5 h). Conformal coverage falls to 0.62 / 0.63 / 0.72 / 0.78 / 0.74 (old 0.61–0.75), below nominal, because calibration residuals come from seen routes; early top-1 attribution (0.52, 0.54) is no better than always blaming the port (0.52). Un-calibrated quantile intervals cover better here (0.70–0.78), so route-shift recalibration is needed.

D. Replication over 3 independent simulated worlds (new seeds change congestion paths and delays; same code, no re-tuning): blend MAE 15.97 / 18.06 / 16.74 (mean 16.9 ± 1.1) vs best baseline 25.75 ± 2.0; mean coverage 0.79 ± 0.02; AUC 0.90 ± 0.01; top-1 attribution after customs 0.80 ± 0.04. In one world the blend is no better than the direct model (18.06 vs 17.94 h) and in another the chain-only model is close (17.27 vs 16.74 h): the blend's edge over its components is about one hour and not uniform, whereas the advantage over baselines is large and consistent.

6. Remaining limitations

- All results come from a simulator I wrote with assumed parameters; the models partly learn my rules. Only wind is real. Nothing here establishes accuracy on real shipments; no real-data evaluation was done.
- Simulated congestion/weather are observed with small noise at known times; real feeds are noisier and later. Conformal intervals assume exchangeability and degrade under route shift. Ablations cover only the direct model. The what-if attribution is not validated against true counterfactuals. The 48 h threshold and delay level were set from the simulator's base rate.
- The chain's previous-stage-delay mismatch (Section 4) is unresolved. Next steps: ingest carrier milestone/AIS feeds through the same data contract, calibrate simulator parameters on real distributions, per-route recalibration.

7. Reproduce

`pip install -r requirements.txt` (pinned; Python 3.13) → `python scripts/run_all.py` (simulate, train, evaluate; writes `report/results.md`, `fig_main.png`, `run_metadata.json`; ~1 min; archives previous outputs) → `python scripts/chain_fix_effect.py` → `python scripts/chain_dprev_mismatch.py` → `python -m pytest tests -q` (13 tests) → `PYTHONPATH=src python -m aiqon.predict examples/at_port_extreme.json`. Two consecutive full runs produced byte-identical `results.md`.